DBMS – DATABASE RELATIONSHIP ANALYSIS USING JOINS

Project Title: Etsy – Online Handmade & Vintage Marketplace Database System

Skill Task: DBMS
Week: 8

Objectives

1. Combine Customer, Product, Order and Payment tables.
2. Implement INNER JOIN, LEFT JOIN and RIGHT JOIN.
3. Retrieve complete order details.
4. Display customer purchase history.
5. Generate multi-table reports.

1. Combining Multiple Tables

The Etsy database stores customer, shop, product, order and payment information in separate tables. These tables are linked through primary and foreign keys and can be combined using JOIN operations to retrieve related information.

Parent Table	Child Table	Join Column	Relationship
Customer	Orders	Customer_ID	1 : M
Orders	Order_Item	Order_ID	1 : M
Product	Order_Item	Product_ID	1 : M
Orders	Payment	Order_ID	1 : 1
Shop	Product	Shop_ID	1 : M

*Note: Table and column names below follow the sample schema in the Appendix. If your own Etsy tables use different names (e.g. Order_Items instead of Order_Item), change them in the queries before running in SQL*Plus.*

2. INNER JOIN

INNER JOIN returns only the records that have matching values in both tables.

2.1 Customer and Orders

```
SELECT c.Customer_ID, c.Customer_Name, o.Order_ID, o.Order_Date, o.Total_Amount
FROM Customer c
INNER JOIN Orders o
    ON c.Customer_ID = o.Customer_ID;
```

CUSTOMER_ID	CUSTOMER_NAME	ORDER_ID	ORDER_DATE	TOTAL_AMOUNT
101	Anitha Kumar	201	16-SEP-26	2700
101	Anitha Kumar	204	20-SEP-26	600
102	Rahul	202	15-SEP-26	1800
103	Priya	203	12-SEP-26	3200

This displays customers who have placed orders. Meera (Customer 104) has no order, so she is not shown.

2.2 Orders and Payment

```
SELECT o.Order_ID, o.Order_Date, o.Total_Amount,
       p.Payment_Mode, p.Payment_Amount, p.Payment_Status
FROM Orders o
INNER JOIN Payment p
    ON o.Order_ID = p.Order_ID;
```

ORDER_ID	ORDER_DATE	TOTAL_AMOUNT	PAYMENT_MODE	PAYMENT_AMOUNT	PAYMENT_STATUS
201	16-SEP-26	2700	UPI	2700	Successful
202	15-SEP-26	1800	Card	1800	Successful
203	12-SEP-26	3200	Net Banking	3200	Successful

This displays orders with their corresponding payment details. Order 204 has no payment record yet, so it is excluded.

2.3 Product and Order_Item

```
SELECT p.Product_ID, p.Product_Name,
       oi.Order_ID, oi.Quantity, oi.Unit_Price, oi.Subtotal
FROM Product p
INNER JOIN Order_Item oi
    ON p.Product_ID = oi.Product_ID
ORDER BY oi.Order_ID;
```

PRODUCT_ID	PRODUCT_NAME	ORDER_ID	QUANTITY	UNIT_PRICE	SUBTOTAL
101	Handmade Silver Ring	201	1	1500	1500
111	Macrame Wall Hanging	201	2	600	1200
131	Hand-Painted Art Print	202	1	900	900
141	Handmade Lavender Soap	202	3	300	900
151	Custom Wedding Invitation	203	1	1200	1200
161	Wooden Toy Train	203	2	1000	2000
141	Handmade Lavender Soap	204	2	300	600

This displays products that have been included in orders.

3. LEFT JOIN

LEFT JOIN returns all records from the left table and the matching records from the right table. Unmatched rows show NULL for the right table.

3.1 Display All Customers and Their Orders

```
SELECT c.Customer_ID, c.Customer_Name, o.Order_ID, o.Order_Date, o.Total_Amount
FROM Customer c
LEFT JOIN Orders o
    ON c.Customer_ID = o.Customer_ID;
```

CUSTOMER_ID	CUSTOMER_NAME	ORDER_ID	ORDER_DATE	TOTAL_AMOUNT
101	Anitha Kumar	201	16-SEP-26	2700
101	Anitha Kumar	204	20-SEP-26	600
102	Rahul	202	15-SEP-26	1800
103	Priya	203	12-SEP-26	3200
104	Meera	NULL	NULL	NULL

This displays all customers, including customers who have not placed an order.

3.2 Orders Without Payment

```
SELECT o.Order_ID, o.Order_Date, o.Total_Amount,
       p.Payment_Mode, p.Payment_Status
FROM Orders o
LEFT JOIN Payment p
    ON o.Order_ID = p.Order_ID
WHERE p.Payment_ID IS NULL;
```

ORDER_ID	ORDER_DATE	TOTAL_AMOUNT	PAYMENT_MODE	PAYMENT_STATUS
204	20-SEP-26	600	NULL	NULL

This finds orders that were placed but not yet paid, which is useful for payment follow-up.

4. RIGHT JOIN

RIGHT JOIN returns all records from the right table and the matching records from the left table.

4.1 Display All Orders and Customer Details

```
SELECT c.Customer_ID, c.Customer_Name, o.Order_ID, o.Order_Date, o.Total_Amount
FROM Customer c
RIGHT JOIN Orders o
    ON c.Customer_ID = o.Customer_ID;
```

CUSTOMER_ID	CUSTOMER_NAME	ORDER_ID	ORDER_DATE	TOTAL_AMOUNT
101	Anitha Kumar	201	16-SEP-26	2700
101	Anitha Kumar	204	20-SEP-26	600
102	Rahul	202	15-SEP-26	1800
103	Priya	203	12-SEP-26	3200

This displays all orders along with the matching customer information. Because every order must belong to a customer (foreign key), the result matches the INNER JOIN here.

4.2 Display All Products, Including Unsold Products

```
SELECT p.Product_ID, p.Product_Name, oi.Order_ID, oi.Quantity
FROM Order_Item oi
RIGHT JOIN Product p
    ON p.Product_ID = oi.Product_ID
ORDER BY p.Product_ID;
```

PRODUCT_ID	PRODUCT_NAME	ORDER_ID	QUANTITY
101	Handmade Silver Ring	201	1
111	Macrame Wall Hanging	201	2
131	Hand-Painted Art Print	202	1
141	Handmade Lavender Soap	202	3
141	Handmade Lavender Soap	204	2
151	Custom Wedding Invitation	203	1
161	Wooden Toy Train	203	2
181	Vintage Brass Key	NULL	NULL

This displays every product in the catalogue. Products with NULL order details (e.g. Vintage Brass Key) have never been ordered.

5. Complete Order Details

This query combines Customer, Orders, Order_Item, Product and Payment tables.

```
SELECT c.Customer_ID, c.Customer_Name, o.Order_ID, o.Order_Date,
       p.Product_ID, p.Product_Name, oi.Quantity, oi.Unit_Price, oi.Subtotal,
       py.Payment_Mode, py.Payment_Amount, py.Payment_Status
FROM Customer c
INNER JOIN Orders o
    ON c.Customer_ID = o.Customer_ID
INNER JOIN Order_Item oi
    ON o.Order_ID = oi.Order_ID
INNER JOIN Product p
    ON oi.Product_ID = p.Product_ID
INNER JOIN Payment py
    ON o.Order_ID = py.Order_ID
ORDER BY o.Order_Date DESC;
```

CUST_ID	CUST_NAME	ORDER_ID	ORDER_DATE	PROD_ID	PRODUCT_NAME	QTY	UNIT_PR	SUBTOTAL	PAY_MODE	PAY_AMT	PAY_STATUS
101	Anitha Kumar	201	16-SEP-26	101	Handmade Silver Ring	1	1500	1500	UPI	2700	Successful
101	Anitha Kumar	201	16-SEP-26	111	Macrame Wall Hanging	2	600	1200	UPI	2700	Successful
102	Rahul	202	15-SEP-26	131	Hand-Painted Art Print	1	900	900	Card	1800	Successful
102	Rahul	202	15-SEP-26	141	Handmade Lavender Soap	3	300	900	Card	1800	Successful
103	Priya	203	12-SEP-26	151	Custom Wedding Invitation	1	1200	1200	Net Banking	3200	Successful
103	Priya	203	12-SEP-26	161	Wooden Toy Train	2	1000	2000	Net Banking	3200	Successful

Order 204 is not shown because it has no payment record (INNER JOIN with Payment).

6. Customer Purchase History

This query displays the products purchased by each customer.

```
SELECT c.Customer_ID, c.Customer_Name, o.Order_ID, o.Order_Date,
       p.Product_Name, oi.Quantity, oi.Subtotal
FROM Customer c
INNER JOIN Orders o
    ON c.Customer_ID = o.Customer_ID
INNER JOIN Order_Item oi
    ON o.Order_ID = oi.Order_ID
INNER JOIN Product p
    ON oi.Product_ID = p.Product_ID
ORDER BY c.Customer_ID, o.Order_Date;
```

CUSTOMER_ID	CUSTOMER_NAME	ORDER_ID	ORDER_DATE	PRODUCT_NAME	QUANTITY	SUBTOTAL
101	Anitha Kumar	201	16-SEP-26	Handmade Silver Ring	1	1500
101	Anitha Kumar	201	16-SEP-26	Macrame Wall Hanging	2	1200
101	Anitha Kumar	204	20-SEP-26	Handmade Lavender Soap	2	600
102	Rahul	202	15-SEP-26	Hand-Painted Art Print	1	900
102	Rahul	202	15-SEP-26	Handmade Lavender Soap	3	900
103	Priya	203	12-SEP-26	Custom Wedding Invitation	1	1200
103	Priya	203	12-SEP-26	Wooden Toy Train	2	2000

7. Customer Purchase History with Payment

```
SELECT c.Customer_ID, c.Customer_Name, o.Order_ID, o.Order_Date,
       p.Product_Name, oi.Quantity, oi.Subtotal,
       py.Payment_Mode, py.Payment_Status
FROM Customer c
INNER JOIN Orders o
    ON c.Customer_ID = o.Customer_ID
INNER JOIN Order_Item oi
    ON o.Order_ID = oi.Order_ID
INNER JOIN Product p
    ON oi.Product_ID = p.Product_ID
INNER JOIN Payment py
    ON o.Order_ID = py.Order_ID
ORDER BY c.Customer_ID;
```

CUST_ID	CUSTOMER_NAME	ORDER_ID	ORDER_DATE	PRODUCT_NAME	QTY	SUBTOTAL	PAYMENT_MODE	PAYMENT_STATUS
101	Anitha Kumar	201	16-SEP-26	Handmade Silver Ring	1	1500	UPI	Successful
101	Anitha Kumar	201	16-SEP-26	Macrame Wall Hanging	2	1200	UPI	Successful
102	Rahul	202	15-SEP-26	Hand-Painted Art Print	1	900	Card	Successful
102	Rahul	202	15-SEP-26	Handmade Lavender Soap	3	900	Card	Successful
103	Priya	203	12-SEP-26	Custom Wedding Invitation	1	1200	Net Banking	Successful
103	Priya	203	12-SEP-26	Wooden Toy Train	2	2000	Net Banking	Successful

8. Multi-Table Business Report

The following query generates a complete customer purchase report.

```
SELECT c.Customer_Name,
       COUNT(DISTINCT o.Order_ID) AS Total_Orders,
       SUM(oi.Subtotal)           AS Total_Purchase,
       SUM(oi.Quantity)           AS Total_Products
FROM Customer c
INNER JOIN Orders o
    ON c.Customer_ID = o.Customer_ID
INNER JOIN Order_Item oi
    ON o.Order_ID = oi.Order_ID
GROUP BY c.Customer_Name
ORDER BY Total_Purchase DESC;
```

CUSTOMER_NAME	TOTAL_ORDERS	TOTAL_PURCHASE	TOTAL_PRODUCTS
Anitha Kumar	2	3300	5
Priya	1	3200	3
Rahul	1	1800	4

8.1 Including customers with no purchases (LEFT JOIN)

```
SELECT c.Customer_Name,
       COUNT(DISTINCT o.Order_ID)    AS Total_Orders,
       NVL(SUM(oi.Subtotal), 0)      AS Total_Purchase,
       NVL(SUM(oi.Quantity), 0)      AS Total_Products
FROM Customer c
LEFT JOIN Orders o
    ON c.Customer_ID = o.Customer_ID
LEFT JOIN Order_Item oi
    ON o.Order_ID = oi.Order_ID
GROUP BY c.Customer_Name
ORDER BY Total_Purchase DESC;
```

CUSTOMER_NAME	TOTAL_ORDERS	TOTAL_PURCHASE	TOTAL_PRODUCTS
Anitha Kumar	2	3300	5
Priya	1	3200	3
Rahul	1	1800	4
Meera	0	0	0

9. Payment Transaction Report

```
SELECT c.Customer_Name, o.Order_ID, o.Order_Date, o.Total_Amount,
       py.Payment_Mode, py.Payment_Amount, py.Payment_Status
FROM Customer c
INNER JOIN Orders o
    ON c.Customer_ID = o.Customer_ID
INNER JOIN Payment py
    ON o.Order_ID = py.Order_ID
ORDER BY o.Order_Date DESC;
```

CUSTOMER_NAME	ORDER_ID	ORDER_DATE	TOTAL_AMOUNT	PAYMENT_MODE	PAYMENT_AMOUNT	PAYMENT_STATUS
Anitha Kumar	201	16-SEP-26	2700	UPI	2700	Successful
Rahul	202	15-SEP-26	1800	Card	1800	Successful
Priya	203	12-SEP-26	3200	Net Banking	3200	Successful

10. Etsy-Specific Report: Sales by Shop

Etsy is a marketplace of independent shops, so this report joins Shop, Product and Order_Item with LEFT JOINs to show every shop, including shops with no sales.

```
SELECT s.Shop_Name,
       NVL(SUM(oi.Quantity), 0) AS Units_Sold,
       NVL(SUM(oi.Subtotal), 0) AS Revenue
FROM Shop s
LEFT JOIN Product p
    ON s.Shop_ID = p.Shop_ID
LEFT JOIN Order_Item oi
    ON p.Product_ID = oi.Product_ID
GROUP BY s.Shop_Name
ORDER BY Revenue DESC, s.Shop_Name;
```

SHOP_NAME	UNITS_SOLD	REVENUE
PaperPalette	2	2100
Little Woodworks	2	2000
Lavender Lane	5	1500

SHOP_NAME	UNITS_SOLD	REVENUE
SilverLeaf Studio	1	1500
Knot & Thread	2	1200
Antique Attic	0	0

Conclusion

INNER JOIN returns only matching rows, LEFT JOIN keeps every row from the left table (e.g. all customers or all orders), and RIGHT JOIN keeps every row from the right table (e.g. all products). Combining Customer, Orders, Order_Item, Product and Payment gives complete order details, purchase history and business reports for the Etsy marketplace.

Appendix: Sample Schema and Data (Oracle SQL*Plus)

Run this only if your tables do not already contain similar data. Replace names/IDs with your own Etsy data if needed.

```
CREATE TABLE Customer (Customer_ID NUMBER PRIMARY KEY, Customer_Name VARCHAR2(50),
                       Email VARCHAR2(80));
CREATE TABLE Shop (Shop_ID NUMBER PRIMARY KEY, Shop_Name VARCHAR2(60));
CREATE TABLE Product (Product_ID NUMBER PRIMARY KEY, Product_Name VARCHAR2(80),
                      Shop_ID NUMBER REFERENCES Shop(Shop_ID));
CREATE TABLE Orders (Order_ID NUMBER PRIMARY KEY, Customer_ID NUMBER REFERENCES Customer(Customer_ID),
                     Order_Date DATE, Total_Amount NUMBER, Order_Status VARCHAR2(20));
CREATE TABLE Order_Item (Order_Item_ID NUMBER PRIMARY KEY, Order_ID NUMBER REFERENCES Orders(Order_ID),
                         Product_ID NUMBER REFERENCES Product(Product_ID),
                         Quantity NUMBER, Unit_Price NUMBER, Subtotal NUMBER);
CREATE TABLE Payment (Payment_ID NUMBER PRIMARY KEY, Order_ID NUMBER REFERENCES Orders(Order_ID),
                      Payment_Mode VARCHAR2(20), Payment_Amount NUMBER, Payment_Status VARCHAR2(20));

INSERT INTO Customer VALUES (101,'Anitha Kumar','anitha@mail.com');
INSERT INTO Customer VALUES (102,'Rahul','rahul@mail.com');
INSERT INTO Customer VALUES (103,'Priya','priya@mail.com');
INSERT INTO Customer VALUES (104,'Meera','meera@mail.com');

INSERT INTO Shop VALUES (1,'SilverLeaf Studio');
INSERT INTO Shop VALUES (2,'Knot & Thread');
INSERT INTO Shop VALUES (3,'PaperPalette');
INSERT INTO Shop VALUES (4,'Lavender Lane');
INSERT INTO Shop VALUES (5,'Little Woodworks');
INSERT INTO Shop VALUES (6,'Antique Attic');

INSERT INTO Product VALUES (101,'Handmade Silver Ring',1);
INSERT INTO Product VALUES (111,'Macrame Wall Hanging',2);
INSERT INTO Product VALUES (131,'Hand-Painted Art Print',3);
INSERT INTO Product VALUES (141,'Handmade Lavender Soap',4);
INSERT INTO Product VALUES (151,'Custom Wedding Invitation',3);
INSERT INTO Product VALUES (161,'Wooden Toy Train',5);
INSERT INTO Product VALUES (181,'Vintage Brass Key',6);

INSERT INTO Orders VALUES (201,101,TO_DATE('16-SEP-26','DD-MON-RR'),2700,'Delivered');
INSERT INTO Orders VALUES (202,102,TO_DATE('15-SEP-26','DD-MON-RR'),1800,'Delivered');
INSERT INTO Orders VALUES (203,103,TO_DATE('12-SEP-26','DD-MON-RR'),3200,'Delivered');
INSERT INTO Orders VALUES (204,101,TO_DATE('20-SEP-26','DD-MON-RR'),600,'Pending');

INSERT INTO Order_Item VALUES (1,201,101,1,1500,1500);
INSERT INTO Order_Item VALUES (2,201,111,2,600,1200);
INSERT INTO Order_Item VALUES (3,202,131,1,900,900);
INSERT INTO Order_Item VALUES (4,202,141,3,300,900);
INSERT INTO Order_Item VALUES (5,203,151,1,1200,1200);
INSERT INTO Order_Item VALUES (6,203,161,2,1000,2000);
INSERT INTO Order_Item VALUES (7,204,141,2,300,600);

INSERT INTO Payment VALUES (1,201,'UPI',2700,'Successful');
INSERT INTO Payment VALUES (2,202,'Card',1800,'Successful');
INSERT INTO Payment VALUES (3,203,'Net Banking',3200,'Successful');
COMMIT;
```